JADAVPUR UNIVERSITY

188, Raja S.C. Mallick Road, Kolkata – 700032

Department of Computer Science and Engineering, FET.

MASTER OF COMPUTER APPLICATION

1st Year 2nd Semester

PYTHON ASSIGNMENT

Name: Snehasish Sarkar
Roll No.: 002510503033

PROGRAM 1: Write a prime generator program using only primes and using python loops.
CODE :

```
from typing import Generator

def is_prime(n: int) -> bool:
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

def primes() -> Generator[int]:
    num = 2

    while True:
        if is_prime(num):
            yield num

        num += 1

n = int(input("Enter the no. of primes you want to print: "))
gen = primes()

for _ in range(n):
    print(next(gen), end=" ")
```

Output :

```
Enter the no. of primes you want to print: 20
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71
```

PROGRAM 2 : Write a discount coupon code using dictionary in Python with different rate coupons for each day of the week.
CODE :

```
coupons = {
    "monday": 10,
    "tuesday": 15,
    "wednesday": 20,
    "thursday": 12,
    "friday": 25,
    "saturday": 30,
    "sunday": 35
}
day = input("Enter the day of the week: ").lower()

if day in coupons:
    print("Discount Coupon Rate =", coupons[day], "%")
else:
    print("Invalid day entered")
```

OUTPUT :

```
Enter the day of the week: monday
Discount Coupon Rate = 10 %
```

PROGRAM 3 : Print first 10 odd and even numbers using iterators and compress. You can use duck typing.

CODE :

```
n = int(input("Enter the range: "))


even = iter([i for i in range(n + 1) if i % 2 == 0])
odd = iter([i for i in range(n + 1) if i % 2 != 0])

print("\nUsing iter()")
print("Even | Odd")

while True:
    try:
        print(next(even), "   ", next(odd))
    except StopIteration:
        print("End")
        break


class Even:
    def __init__(self, limit):
        self.item = 0
        self.limit = limit

    def __iter__(self):
        return self

    def __next__(self):
        if self.item > self.limit:
            raise StopIteration

        value = self.item
        self.item += 2
        return value

class Odd:
    def __init__(self, limit):
        self.item = 1
        self.limit = limit

    def __iter__(self):
        return self

    def __next__(self):
        if self.item > self.limit:
            raise StopIteration

        value = self.item
        self.item += 2
        return value

print("\nUsing Custom Iterators")
```

```
print("Even | Odd")

e = Even(n)
o = Odd(n)

for i, j in zip(e, o):
    print(i, j)
```

OUTPUT :

```
Enter the range: 10

Using iter()
Even | Odd
0    1
2    3
4    5
6    7
8    9
End

Using Custom Iterators
Even | Odd
0 1
2 3
4 5
6 7
8 9
```

PROGRAM 4: Write a regular expression to validate a phone number.

CODE :

```
import re

phone_re = r'^(\+91\s?)?[6-9]\d{4}\s?\d{5}$'

while True:
    phone = input("Enter phone number: ")

    if re.fullmatch(phone_re, phone):
        print("Valid")
    else:
        print("Invalid")
```

OUTPUT :

```
Enter phone number: 8927043274
Valid
Enter phone number: +918927043274
Valid
Enter phone number: +91 89270 43274
Valid
Enter phone number: 89270 43274
Valid
Enter phone number: 0342453523
Invalid
Enter phone number: 19304829458
Invalid
Enter phone number: 782 89 423 45
Invalid
```

PROGRAM 5: Write first seven Fibinacci numbers using generator next function/ yield in python. Trace and memorize the function.

CODE :

```
from typing import Generator
def fibonaccci() -> Generator[int]:
    n0 = 0
    n1 = 1
    yield n0
    yield n1

    while True:
        n2 = n0 + n1
        yield n2
        n0 = n1
        n1 = n2

fib = fibonaccci()

n = int(input("Enter the number : "))
for _ in range(n):
    print(next(fib), end=" ")
```

OUTPUT :

```
Enter the number : 10
0 1 1 2 3 5 8 13 21 34
```

PROGRAM 6: Write a simple program which loops over a list of user data (tuples containing a username, email and age) and adds each user to a directory if the user is at least 16 years old. You do not need to store the age. Write a simple exception hierarchy which defines a different exception for each of these error conditions:

- **the username is not unique**
- **the age is not a positive integer**
- **the user is under 16**
- **the email address is not valid (a simple check for a username, the @ symbol and a domain name is sufficient)**

Raise these exceptions in your program where appropriate. Whenever an exception occurs, your program should move onto the next set of data in the list. Print a different error message for each different kind of exception.

CODE :

```
class UserError(Exception):
    pass
class UsernameNotUniqueError(UserError):
    pass
class InvalidAgeError(UserError):
    pass
class UnderAgeError(UserError):
    pass
class InvalidEmailError(UserError):
    pass
users = [
    ("rahul", "rahul@gmail.com", 20),
    ("amit", "amit@yahoo.com", 17),
    ("rahul", "rahul2@gmail.com", 25),
    ("sita", "sita.gmail.com", 22),
    ("gita", "gita@gmail.com", -5),
    ("rani", "rani@gmail.com", 15),
    ("soham", "soham@gmail.com", 18)
]
directory = {}
usernames = set()


for username, email, age in users:
    try:
        if username in usernames:
            raise UsernameNotUniqueError(
                f"Username '{username}' already exists."
            )
        if not isinstance(age, int) or age <= 0:
            raise InvalidAgeError(
                f"Invalid age '{age}'. Age must be positive."
            )
        if age < 16:
            raise UnderAgeError(
                f"User '{username}' is under 16."
            )
        if "@" not in email or "." not in email.split("@")[-1]:
            raise InvalidEmailError(
                f"Invalid email '{email}'."
            )
```

```
        directory[username] = {
            "email": email,
            "age": age
        }

        usernames.add(username)
        print(f"{username} added successfully.")
    except UsernameNotUniqueError as e:
        print("Username Error :", e)
    except InvalidAgeError as e:
        print("Age Error      :", e)
    except UnderAgeError as e:
        print("Under Age Error:", e)
    except InvalidEmailError as e:
        print("Email Error    :", e)
print("\nValid Users Directory")
print(directory)
```

OUTPUT :

```
rahul added successfully.
amit added successfully.
Username Error : Username 'rahul' already exists.
Email Error    : Invalid email 'sita.gmail.com'.
Age Error      : Invalid age '-5'. Age must be positive.
Under Age Error: User 'rani' is under 16.
soham added successfully.

Valid Users Directory
{'rahul': {'email': 'rahul@gmail.com', 'age': 20}, 'amit': {'email': 'amit@yahoo.com', 'age': 17}, 'soham': {'email': 'soham@gmail.com', 'age': 18}}
```

PROGRAM 7: Write a function findfiles that recursively descends the directory tree for the specified directory and generates paths of all the files in the tree.

CODE :

```
import o
def findfiles(path):
    for item in os.listdir(path):
        full_path = os.path.join(path, item)
        if os.path.isfile(full_path):
            yield full_path
        elif os.path.isdir(full_path):
            yield from findfiles(full_path)
directory = r'C:\Users\sneha\Desktop\Github\Assignments\python'
for file in findfiles(directory):
    print(file)
```

OUTPUT :

```
C:\Users\sneha\Desktop\Github\Assignments\python\folder\test.txt
C:\Users\sneha\Desktop\Github\Assignments\python\program1.py
C:\Users\sneha\Desktop\Github\Assignments\python\program10.py
C:\Users\sneha\Desktop\Github\Assignments\python\program11.py
C:\Users\sneha\Desktop\Github\Assignments\python\program12.py
C:\Users\sneha\Desktop\Github\Assignments\python\program13.py
C:\Users\sneha\Desktop\Github\Assignments\python\program14.py
C:\Users\sneha\Desktop\Github\Assignments\python\program15.py
C:\Users\sneha\Desktop\Github\Assignments\python\program2.py
C:\Users\sneha\Desktop\Github\Assignments\python\program3.py
C:\Users\sneha\Desktop\Github\Assignments\python\program4.py
C:\Users\sneha\Desktop\Github\Assignments\python\program5.py
C:\Users\sneha\Desktop\Github\Assignments\python\program6.py
C:\Users\sneha\Desktop\Github\Assignments\python\program7.py
C:\Users\sneha\Desktop\Github\Assignments\python\program8.py
C:\Users\sneha\Desktop\Github\Assignments\python\program9.py
```

PROGRAM 8: Create a list of all the numbers up to N=50 which are multiples of five using anonymous function.

CODE :

```
n = int(input("Enter the number : "))
l = list(filter(lambda x : x % 5 == 0, range(n + 1)))
print(l)
```

OUTPUT :

```
Enter the number : 100
[0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100]
```

PROGRAM 9: Use map and zip to find the element-wise maximum amongst sequences of student list, subject list and marks list.

CODE :

```
import random

students = ["Rahul", "Amit", "Sita", "Rani", "Priya"]
subjects = ["Math", "Physics", "Chemistry", "Biology", "English"]
marks = [random.randint(0, 100) for _ in range(5)]

print("Students :", students)
print("Subjects :", subjects)
print("Marks    :", marks)


result = list(map(lambda t: max(t, key=str), zip(students, subjects, marks)))

print("\nElement-wise Maximum:")
print(result)
```

OUTPUT :

```
Students : ['Rahul', 'Amit', 'Sita', 'Rani', 'Priya']
Subjects : ['Math', 'Physics', 'Chemistry', 'Biology', 'English']
Marks    : [20, 17, 23, 96, 14]

Element-wise Maximum:
['Rahul', 'Physics', 'Sita', 'Rani', 'Priya']
```

PROGRAM 10: Filter out the odd squares using map, filter, list.

CODE :

```
import random as r
l = list(filter(lambda x : x % 2 == 1, map(lambda x : x ** 2, range(r.randint(0, 100)))))
print(l)
```

OUTPUT :

```
[1, 9, 25, 49, 81, 121, 169, 225, 289, 361, 441]
```

PROGRAM 11: Let's find all Pythagorean triples whose short sides are numbers smaller than 10. use filter and comprehension.

CODE :

```
import random as r

n = r.randint(1, 100)
triples = [
    (a, b, int((a**2 + b**2)**0.5))
    for a in range(1, n)
    for b in range(a, n)
    if ((a**2 + b**2)**0.5).is_integer()
]

t = [(a, b, int((a**2 + b**2)**0.5)) for a, b in filter(lambda pair:
((pair[0]**2 + pair[1]**2)**0.5).is_integer(), [(x, y) for x in range(1, n) for
y in range(x, n)])]


print("List Comprehension Result:")
print(triples)
print("\nFilter Result:")
print(t)
```

OUTPUT :

```
List Comprehension Result:
[(3, 4, 5), (5, 12, 13), (6, 8, 10), (8, 15, 17), (9, 12, 15), (12, 16, 20), (15, 20, 25), (20, 21, 29)]

Filter Result:
[(3, 4, 5), (5, 12, 13), (6, 8, 10), (8, 15, 17), (9, 12, 15), (12, 16, 20), (15, 20, 25), (20, 21, 29)]
```

PROGRAM 12: Enumerate the sequence of all lowercase ASCII letters, starting from 1, using enumerate.

CODE :

```
import string

lowercase_letters = string.ascii_lowercase

enumerated_letters = list(enumerate(lowercase_letters, start=1))

print(enumerated_letters)
```

OUTPUT :

```
[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e'), (6, 'f'), (7, 'g'), (8, 'h'),
(9, 'i'), (10, 'j'), (11, 'k'), (12, 'l'), (13, 'm'), (14, 'n'), (15, 'o'), (16,
'p'), (17, 'q'), (18, 'r'), (19, 's'), (20, 't'), (21, 'u'), (22, 'v'), (23, 'w')
, (24, 'x'), (25, 'y'), (26, 'z')]
```

PROGRAM 13: Write a code which yields all terms of the geometric progression a, aq, aq2, aq3, ... When the progression produces a term that is greater than 100,000, the generator stops (with a return statement). Compute total time and time within the loop.

CODE :

```
import random as r

def geometric_progress(a, q):
    yield a
    i = 1
    while i < 100000:
        yield a * q ** i

        i += 1
    return None

gp = geometric_progress(r.randint(1, 10), r.randint(1, 10))

for _ in range(r.randint(1, 100)):
    print(next(gp))
```

OUTPUT :

```
4
16
64
256
1024
4096
16384
65536
262144
```

PROGRAM 14: Search for palindrome and unique words in a text using class method and string method.

CODE :

```
class TextAnalyzer:
    def __init__(self, text):
        self.text = text

    @classmethod
    def is_palindrome(cls, word):
        """Class method to check if a single word reads the same backwards."""
        cleaned_word = word.lower()
        return cleaned_word == cleaned_word[::-1] and len(cleaned_word) > 1

    def find_unique_and_palindromes(self):
        """String method that processes the instance's text."""
        cleaned_text = "".join(char.lower() for char in self.text if
char.isalnum() or char.isspace())

        words = cleaned_text.split()
        unique_words = set(words)

        palindromes = [word for word in unique_words if
self.is_palindrome(word)]

        return list(unique_words), palindromes


sample_text = "Madam went to buy a racecar, but she forgot her wallet. Wow,
what a day!"

analyzer = TextAnalyzer(sample_text)
unique_list, palindrome_list = analyzer.find_unique_and_palindromes()

print("--- Analysis Results ---")
print(f"Unique Words Found:\n{unique_list}\n")
print(f"Palindromes Found:\n{palindrome_list}")
```

OUTPUT :

```
--- Analysis Results ---
Unique Words Found:
['day', 'buy', 'a', 'madam', 'forgot', 'wallet', 'went', 'racecar', 'her', 'she', 'to', 'but', 'wow', 'what']

Palindromes Found:
['madam', 'racecar', 'wow']
```

PROGRAM 15: Create a BankAccount class. Your class should support these methods: deposit, withdraw, get_balance, change_pin. Create one SavingsAccount class that behaves just like a BankAccount class, but also has an interest rate and a method that increases the balance by the appropriate amount of interest. Create another FeeSavingsAccount class that behaves just like a SavingsAccount, but also charges a fee every time you withdraw money. The fee should be set in the constructor and deducted before each withdrawal.

CODE :

```
import random as r

class BankAccount:
    def __init__(self, name, mobile, pin):
        self.name = name
        self.mobile = mobile
        self.balance = 0.0
        self.account = str(r.randint(100000000, 999999999))
        self.pin = pin

    def deposit(self, amount, pin):
        if pin != self.pin:
            raise Exception("Pin not matched.")

        self.balance += amount
        return f"Deposited ₹{amount}. Balance = ₹{self.balance:.2f}"

    def withdraw(self, amount, pin):
        if pin != self.pin:
            raise Exception("Pin not matched.")

        if amount > self.balance:
            raise Exception("Insufficient balance.")

        self.balance -= amount
        return f"Withdrawn ₹{amount}. Balance = ₹{self.balance:.2f}"

    def get_balance(self, pin):
        if pin != self.pin:
            raise Exception("Pin not matched.")

        return self.balance

    def change_pin(self, old_pin, new_pin):
        if old_pin != self.pin:
            raise Exception("Pin not matched.")

        self.pin = new_pin
        return "PIN changed successfully."


class SavingsAccount(BankAccount):
    def __init__(self, name, mobile, pin, interest_rate):
        super().__init__(name, mobile, pin)
        self.interest_rate = interest_rate

    def add_interest(self):
```

```
        interest = self.balance * self.interest_rate / 100
        self.balance += interest

        return (f"Interest Added = ₹{interest:.2f}, "
                f"New Balance = ₹{self.balance:.2f}")


class FeeSavingsAccount(SavingsAccount):
    def __init__(self, name, mobile, pin, interest_rate, fee):
        super().__init__(name, mobile, pin, interest_rate)
        self.fee = fee

    def withdraw(self, amount, pin):
        total = amount + self.fee

        if pin != self.pin:
            raise Exception("Pin not matched.")

        if total > self.balance:
            raise Exception("Insufficient balance.")

        self.balance -= total

        return (f"Withdrawn ₹{amount}, "
                f"Fee ₹{self.fee}, "
                f"Balance = ₹{self.balance:.2f}")



acc = FeeSavingsAccount(
    "Rahul",
    "9876543210",
    1234,
    5,
    10
)

print(acc.deposit(5000, 1234))
print(acc.add_interest())
print(acc.withdraw(1000, 1234))
print("Balance =", acc.get_balance(1234))
print(acc.change_pin(1234, 4321))
```

OUTPUT :

```
Deposited ₹5000. Balance = ₹5000.00
Interest Added = ₹250.00, New Balance = ₹5250.00
Withdrawn ₹1000, Fee ₹10, Balance = ₹4240.00
Balance = 4240.0
PIN changed successfully.
```

PROGRAM 16: Write an operator overloading for len which shows string length for any given string and return only length of repetitive words with the text if the text has some repetitive parts. Determine the most frequently occurring words using most_common.

CODE :

```
from collections import Counter

class TextAnalyzer:
    def __init__(self, text):
        self.text = text
        self.words = text.split()

    def __len__(self):
        freq = Counter(self.words)

        repeated_words = [word for word, count in freq.items()
                          if count > 1]

        if repeated_words:
            return sum(len(word) for word in repeated_words)

        return len(self.text)

    def most_frequent(self):
        freq = Counter(self.words)
        return freq.most_common()



text = input("Enter a text: ")

obj = TextAnalyzer(text)

print("\nLength using len():", len(obj))

print("\nWord Frequencies:")
for word, count in obj.most_frequent():
    print(f"{word} -> {count}")
```

OUTPUT :

```
Enter a text: cat dog cat bird dog cat

Length using len(): 6

Word Frequencies:
cat -> 3
dog -> 2
bird -> 1
```

PROGRAM 17: Implement a priority queue that sorts items by a given priority and always returns the item with the highest priority on each pop operation.

CODE :

```
class PriorityQueue:
    def __init__(self):
        self.queue = []

    def push(self, item, priority):
        self.queue.append((priority, item))
        self.queue.sort(reverse=True)  # Highest priority first

    def pop(self):
        if self.is_empty():
            raise Exception("Priority Queue is empty")

        priority, item = self.queue.pop(0)
        return item

    def peek(self):
        if self.is_empty():
            raise Exception("Priority Queue is empty")

        return self.queue[0][1]

    def is_empty(self):
        return len(self.queue) == 0

    def display(self):
        print(self.queue)


# Driver Code
pq = PriorityQueue()

pq.push("Task A", 2)
pq.push("Task B", 5)
pq.push("Task C", 1)
pq.push("Task D", 4)

print("Queue:")
pq.display()

print("\nPop Operations:")
while not pq.is_empty():
    print(pq.pop())
```

OUTPUT :

```
Queue:
[(5, 'Task B'), (4, 'Task D'), (2, 'Task A'), (1, 'Task C')]

Pop Operations:
Task B
Task D
Task A
Task C
```

PROGRAM 18: Make a list of the largest or smallest N items in a collection.
CODE :

```
import random
import heapq

nums = [random.randint(1, 100) for _ in range(10)]

print("List:", nums)

n = int(input("Enter N: "))

print("Largest N items:", heapq.nlargest(n, nums))
print("Smallest N items:", heapq.nsmallest(n, nums))
```

OUTPUT :

```
List: [100, 7, 76, 33, 4, 3, 98, 91, 63, 34]
Enter N: 5
Largest N items: [100, 98, 91, 76, 63]
Smallest N items: [3, 4, 7, 33, 34]
```

PROGRAM 19: Create a dictionary that maps stock names to prices, which will keep insertion order. Find minimum price, maximum price and sort items according to their prices in first dictionary. Create another second stock dictionary. Find items that are only in first dictionary and find items whose prices do not match. Remove duplicate items from first dictionary. Sort both dictionaries for incrementing prices. Group items in first dictionary by price in multiple of 500. Find an item with price=800 from both dictionaries.

CODE :

```
from collections import defaultdict
stock1 = {
    "TCS": 3500,
    "INFY": 1800,
    "WIPRO": 500,
    "HCL": 1500,
    "TECHM": 1800,
    "LT": 3500,
    "SBI": 800
}

stock2 = {
    "TCS": 3500,
    "INFY": 2000,
    "WIPRO": 500,
    "RELIANCE": 3000,
    "SBI": 800,
    "ICICI": 1200
}

print("Stock Dictionary 1:")
print(stock1)

print("\nStock Dictionary 2:")
print(stock2)

print("\nMinimum Price:", min(stock1.values()))
print("Maximum Price:", max(stock1.values()))

print("\nStock1 Sorted by Price:")
sorted_stock1 = dict(sorted(stock1.items(), key=lambda x: x[1]))
print(sorted_stock1)

only_first = stock1.keys() - stock2.keys()

print("\nItems only in Stock1:")
for item in only_first:
    print(item, ":", stock1[item])

print("\nItems with Different Prices:")

for key in stock1.keys() & stock2.keys():
    if stock1[key] != stock2[key]:
        print(key, "->", stock1[key], stock2[key])

unique_stock = {}
```

```
for k, v in stock1.items():
    if v not in unique_stock.values():
        unique_stock[k] = v

print("\nStock1 after Removing Duplicate Prices:")
print(unique_stock)

print("\nStock1 Sorted:")
print(dict(sorted(stock1.items(), key=lambda x: x[1])))

print("\nStock2 Sorted:")
print(dict(sorted(stock2.items(), key=lambda x: x[1])))

groups = defaultdict(list)

for stock, price in stock1.items():
    group = (price // 500) * 500
    groups[group].append(stock)

print("\nGrouped by Price Multiples of 500:")

for group, stocks in groups.items():
    print(f"{group}-{group+499} :", stocks)

print("\nItems with Price = 800")

for stock, price in stock1.items():
    if price == 800:
        print("Stock1 ->", stock)

for stock, price in stock2.items():
    if price == 800:
        print("Stock2 ->", stock)
```

OUTPUT :

```
Stock Dictionary 1:
{'TCS': 3500, 'INFY': 1800, 'WIPRO': 500, 'HCL': 1500, 'TECHM': 1800, 'LT': 3500, 'SBI': 800}

Stock Dictionary 2:
{'TCS': 3500, 'INFY': 2000, 'WIPRO': 500, 'RELIANCE': 3000, 'SBI': 800, 'ICICI': 1200}

Minimum Price: 500
Maximum Price: 3500

Stock1 Sorted by Price:
{'WIPRO': 500, 'SBI': 800, 'HCL': 1500, 'INFY': 1800, 'TECHM': 1800, 'TCS': 3500, 'LT': 3500}

Items only in Stock1:
TECHM : 1800
LT : 3500
HCL : 1500

Items with Different Prices:
INFY -> 1800 2000

Stock1 after Removing Duplicate Prices:
{'TCS': 3500, 'INFY': 1800, 'WIPRO': 500, 'HCL': 1500, 'SBI': 800}

Stock1 Sorted:
{'WIPRO': 500, 'SBI': 800, 'HCL': 1500, 'INFY': 1800, 'TECHM': 1800, 'TCS': 3500, 'LT': 3500}

Stock2 Sorted:
{'WIPRO': 500, 'SBI': 800, 'ICICI': 1200, 'INFY': 2000, 'RELIANCE': 3000, 'TCS': 3500}

Grouped by Price Multiples of 500:
3500-3999 : ['TCS', 'LT']
1500-1999 : ['INFY', 'HCL', 'TECHM']
500-999 : ['WIPRO', 'SBI']

Items with Price = 800
Stock1 -> SBI
Stock2 -> SBI
```

PROGRAM 20: Write a function that flattens a nested dictionary structure like one obtain from Twitter and Facebook APIs or from some JSON file.

```
nested = {
   'fullname': 'Alessandra',
   'age': 41,
   'phone-numbers': ['+447421234567', '+447423456789'],
   'residence': {
      'address': {
         'first-line': 'Alexandra Rd',
         'second-line': '',
      },
      'zip': 'N8 0PP',
      'city': 'London',
      'country': 'UK',
   },
}
```

CODE :

```
def flatten_dict(d, parent_key='', sep='.'):
    result = {}
    for key, value in d.items():
        new_key = parent_key + sep + key if parent_key else key
        if isinstance(value, dict):
            result.update(flatten_dict(value, new_key, sep))
        else:
            result[new_key] = value
    return result
nested = {
    'fullname': 'Alessandra',
    'age': 41,
    'phone-numbers': ['+447421234567', '+447423456789'],
    'residence': {
        'address': {
            'first-line': 'Alexandra Rd',
            'second-line': '',
        },
        'zip': 'N8 0PP',
        'city': 'London',
        'country': 'UK',
    },
}
flat = flatten_dict(nested)
print("Flattened Dictionary:")
for k, v in flat.items():
    print(f"{k} : {v}")
```

OUTPUT :

```
Flattened Dictionary:
fullname : Alessandra
age : 41
phone-numbers : ['+447421234567', '+447423456789']
residence.address.first-line : Alexandra Rd
residence.address.second-line :
residence.zip : N8 0PP
residence.city : London
residence.country : UK
```